

Rapport 1er semaine du Pharo

Avant de commencer les exercices, j'ai d'abord pris le temps de découvrir les bases du **langage Pharo** et de comprendre son fonctionnement général.

J'ai notamment appris que, dans Pharo, **presque tout est un objet**. On ne pense pas principalement : « J'appelle une fonction. » Mais plutôt : « **J'envoie un message à un objet.** »

J'ai également appris les variables et l'affectation :

```
| age |  
age := 20.
```

J'ai découvert les opérateurs comme +, -, etc., ainsi que les chaînes de caractères qui s'écrivent avec des apostrophes.

J'ai aussi appris les deux types de messages : **les messages sans argument et les messages avec argument.**

Pour les conditions, j'ai découvert qu'en Pharo on peut utiliser :

```
age >= 18  
    ifTrue: [ 'majeur' ]  
    ifFalse: [ 'mineur' ]
```

au lieu d'utiliser la syntaxe classique if / else. J'ai également découvert les boucles, notamment avec **to:** et **do:.**

Exercice 1

Dans le premier exercice, j'ai appris plusieurs fonctionnalités importantes de l'environnement Pharo ainsi que des bases du langage.

Le Playground

J'ai appris qu'il est possible d'ouvrir directement un nouveau Playground avec :

```
StPlaygroundPresenter open.
```

Cela permet de créer un nouveau Playground sans passer par le Browser.

Action	Fonction
Do It	Exécute le code sélectionné
Print It	Exécute le code et affiche le résultat
Inspect It	Exécute le code et ouvre un inspecteur sur l'objet

J'ai aussi appris les raccourcis clavier, notamment **Ctrl/Cmd + D** pour Do It. Une autre fonctionnalité pratique est que **double-cliquer au début d'une ligne permet de sélectionner toute la ligne.**

Le point .

J'ai appris que le point . sert à séparer les instructions. Il signifie essentiellement : « **Cette instruction est terminée, je passe à la suivante.** »

```
instruction1.  
instruction2.
```

Les caractères et les chaînes de caractères

J'ai appris qu'un caractère peut être créé avec l'opérateur `$`. J'ai également découvert la classe **Character** et des messages permettant d'obtenir certains caractères spéciaux, comme **Character.cr** pour un retour à la ligne.

Pour les chaînes de caractères, j'ai découvert plusieurs méthodes utiles, comme **size** pour obtenir la longueur, **uppercase** pour convertir en majuscules, ainsi que la concaténation avec `,`.

Exercice 2

Le deuxième exercice m'a permis d'aller plus loin dans Pharo et de créer une vraie classe. J'ai appris à créer une **classe**, un **package**, des **variables d'instance** ainsi que des **méthodes**.

Création d'une classe et d'un package

J'ai créé une classe **Counter** avec une variable d'instance **count**. J'ai ainsi appris à comprendre le rôle de la classe, des variables d'instance et du package.

Méthodes d'instance et méthodes de classe

J'ai appris la différence entre une **méthode d'instance** et une **méthode de classe**. Une méthode d'instance est envoyée à un objet, alors qu'une méthode de classe est envoyée à la classe.

J'ai également mieux compris le rôle de **self** : dans une méthode d'instance, il représente l'objet courant ; dans une méthode de classe, il représente la classe courante.

La méthode initialize

J'ai appris que **initialize** sert à donner un **état initial à un objet**. Elle est exécutée automatiquement lors de la création normale d'un objet avec **new**.

Cela m'a permis de comprendre la différence entre **initialize** et les méthodes ordinaires, qui doivent être appelées explicitement.

Les tests

J'ai appris à créer des **tests automatisés** afin de vérifier que les méthodes de ma classe fonctionnent correctement. Par exemple, j'ai réalisé un test pour vérifier l'incrémement d'un compteur :

```
testIncrement
| c |
c := Counter new.
c count: 3.
c increment.
self assert: c count equals: 4
```

printOn:

J'ai également appris la méthode **printOn:**. Elle permet de **définir la représentation textuelle d'un objet**. J'ai compris qu'elle utilise un stream pour construire cette représentation et que **printString** permet de transformer une valeur en chaîne de caractères.

Utilisation d'Iceberg et de GitHub

J'ai appris à utiliser **Iceberg**, l'outil intégré à Pharo qui permet de gérer **Git directement depuis Pharo**. Cela m'a permis de travailler avec Git sans utiliser les commandes Git dans le terminal.

J'ai appris à sauvegarder mes modifications, faire des commits, créer et utiliser un repository GitHub, connecter mon projet Pharo à GitHub et envoyer mon projet sur GitHub avec un push.

J'ai donc créé un repository GitHub pour mon projet **MyCounter** et j'ai effectué le push directement depuis Pharo avec Iceberg.

Difficultés rencontrées et comment je les ai surmontées

La principale difficulté que j'ai rencontrée était que **le push vers GitHub ne fonctionnait pas**. J'ai cherché l'origine du problème et j'ai compris qu'il s'agissait d'un problème lié à l'authentification GitHub.

Pour résoudre ce problème, j'ai **généralisé un Personal Access Token (PAT) sur GitHub** et je l'ai utilisé pour l'authentification à la place de mon mot de passe habituel. Après avoir créé et configuré le token correctement, le problème a été résolu et j'ai réussi à effectuer le push de mon projet vers GitHub.